



Pet Adoption System

Dr. Youssef Sobhy

Team ID:

بولا صابر كيرلس لبيب

20231700211

محمد سامح السيد
عبدالرحمن

20231700244

أسماء وليد سيد معوض

20231700208

زياد احمد حسين

20231700224

طه مصطفى عبد الفتاح
شحاتة

20231700229

Pet System Adoption:

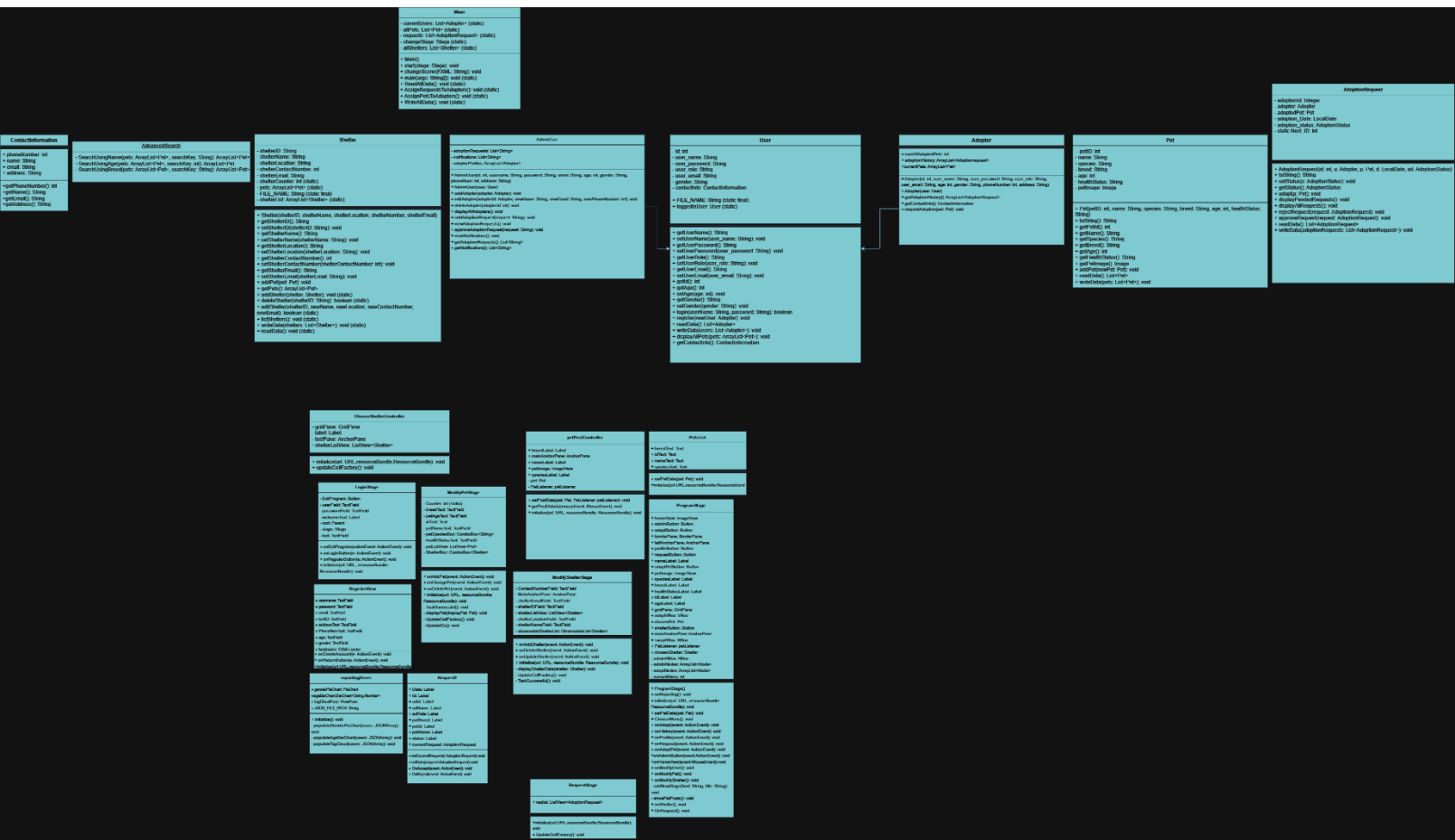
Project Overview

The Pet Adoption System aims to simplify the process of finding homes for pets and connecting adopters with pets in need. The system is designed using Object-Oriented Programming (OOP) principles to ensure scalability, maintainability, and reusability.

The system consists of 8 essential classes (Console-based) beside the Main Class and GUI:

- User (Abstract)
- -----adopter (Inherits from User)
- -----AdminUser (Inherits from User)
- AdoptionRequest
- AdvancedSearch
- ContactInformation
- Pet
- Shelter

A quick view over each Class and methods



○ **User Class : (abstract)**

Overview:

Abstract class represents a generic user with attributes and methods for handling user operations. Throws and handles exceptions Writing and Reading data from JSON.

- ID
- Email
- Password
- Role
- Object from ContactInfo (Aggregation)
- Name
- Age
- Gender

Methods:

- getters and setters
- Login& Register
- Read & Write

Abstract Methods:

- getCurrentPets
 - getNumOfadoptedPets
 - getAdoptionHistory
-
-

Project Overflow

User Registration and Authentication:

- Roles: Adopter, Admin.

Steps:

- Registration: The user creates an account by providing:
 - Name, email, phone number, and password.
- Login: The user signs in with their Data (email and password).

Validation:

- The system is checking within the stored data.(JSON file)
- Depending on the role:

Adopter: Displays available shelters and pets.

Admin: Displays pending adoption requests and system statistics with the ability to modify users, shelter, and pets.

- Input: User credentials.
- Output: Access to the system or error messages for incorrect credentials.

2. Viewing Shelters

- **For both:** Adopter, Admin.
- **Steps:**

1. After login, the **Adopter** views a list of shelters.
 2. Each shelter includes:
 - Shelter Name.
 - Location.
 - Contact Information.
 - Number of pets available.
 3. Adopter selects a shelter to view its pets.
 4. **Admin** can also view all shelters and their Statistics with the ability to add, edit or delete a shelter.
- **Input:** Adopter selects a shelter from the list.
 - **Output:** Display of pets available in the selected shelter.

3.Home Page

Description:

The home page serves as the central dashboard for both **users** and **admins**, providing tailored functionality based on their role in the system.

For Users:

Users interact with the following features:

1. **Statistics Display:**

- Users can view summarized statistics, such as:
 - Gender distribution of adopters.
 - Most adopted type of pets.
 - Locations and time of adoption.

2. **Adoption Functionality:**

- Users can browse available pets categorized by shelters and species.
- Search functionality allows users to filter pets based on:
 - Name.
 - Age.
 - Breed.
- Clicking the **Adopt** button sends a request that is pending approval by an admin.

3. **Profile Management:**

- Users can edit their personal details for Ex. Name or email
- View their **adoption history**, all the adoption requests are either approved, pended or denied.
- View their **current pets**, displayed in a card layout with the following details:
 - Name.
 - Breed.
 - Age.

For Admins:

Admins have all the functionalities available to users, in addition to **admin-F functionalities** to manage the system:

1. User Management:

- Modify user profiles, including Add new users, modify existing user, or removing users.

2. Pet Management:

- Add, remove, or update pets data from the system.
- Assign pets to specific shelters.

3. Shelter Management:

- Modify shelter details, such as:
 - Name.
 - Location.
 - Contact information.

4. Pending Requests:

- Admins can review adoption requests sent by users.
- Requests can be:
 - **Approved:** The pet is assigned to the user, and the user's current pets count is updated. And then the pet disappears from the shelter
 - **Denied:** The user is notified, and the request is removed.

4. Viewing & adopting Pets

Steps:

1. Adopter views pets in the selected shelter.
2. Details displayed for each pet:
 - Name, age, breed, health status, adoption status.
 - **Advanced Search:** Filters pets based on age, breed, or species
3. Adopter selects a pet and submits an adoption request.

Request includes:

- Adopter ID, Pet ID, and date.
4. Request is stored with a **Pending** status.
-
- **Input:** Adopter selects a pet.
 - **Output:** Adoption request is sent to admin waiting to approve or deny.

5. Admin Approving or Rejecting Requests

- **Steps:**
 1. Admin views the list of pending adoption requests.
 2. For each request, Admin reviews:

- Adopter details.
- Pet details.

3. Admin approves or rejects the request:

- **Approve:** Marks request as approved, updates the adopter's currentPets, and sends a notification.
 - **Reject:** Marks request as rejected and the user cant adopt that pet again.
- **Input:** Admin selects a request and performs an action (Approve/Reject).
 - **Output:**
 - Notification sent to the adopter.
 - Pet adoption status updated in the system.
 - Update statistics in the system after approving the request.

6. Notifications

- **Steps:**
 1. Adopters receive notifications:
 - When their adoption request is approved or rejected.
 2. Notifications can be displayed in: Once the user login again as a pop up message at the bottom of the screen that lasts for 5 seconds.
- **Input:** Admin deny or approve request.
- **Output:** Displaying a notification when logged in.

Summary of the Project:

The project handles correctly the 4 main principles of the OOP

1. Inheritance:

- Classes inherit shared behaviors and properties to minimize redundancy.
- Example:
 - A User class acts as a parent for Admin and RegularUser subclasses, which inherit common attributes like name, email, and methods such as login() and viewProfile().

2. Encapsulation:

- Access to sensitive data is controlled using getter and setter methods.
- Example:
 - Variables like password and userID are marked as private and accessed through methods like getPassword() or setPassword().

3. Polymorphism:

- Enables dynamic behavior for similar actions across different classes.
- Example:
 - The viewDashboard() method is implemented differently in Admin and RegularUser classes.
 - Override : like getCurrentPets which display current pets for user but display all pets for Admins.

4. Abstraction:

- Focuses on essential details while hiding implementation complexities.
- Example:

- The User abstract class defines common behaviors like `getCurrentPets` which display current pets for user but display all pets for Admins.
-

Exception Handling:

The system handles errors effectively, ensuring smooth operation by:

1. Throwing Existing Exceptions:

- Example:
 - Throwing an `InvalidInputException` when user input does not meet predefined criteria.

2. Custom Exceptions:

- Example:
 - Creating a `RequestPendingException` to notify users when they attempt duplicate adoption requests for the same pet.
-

Handling Variables and Methods:

1. Static Methods:

- Used for utility functions shared across the system, such as generating unique user IDs or calculating adoption statistics.

2. Abstract Methods:

- Defined in abstract classes to enforce implementation in subclasses.

3. Variable Access Levels:

- **Public:** Accessible across the system, e.g., pet names.
- **Private:** Restricted access, e.g., user passwords.
- **Static Variables:** Shared across instances, e.g., a counter for the total number of pets in the system.

Data Persistence:

To ensure continuity and manage data efficiently, the system saves:

1. **Shelters:**

- Information about shelter locations, contact details, and pets available.

2. **Requests:**

- Pending, approved, and denied adoption requests.

3. **Users:**

- User details, adoption history, and current pets.

Data is stored in **JSON files**, enabling easy access, updates, and compatibility with other systems.